



**BERLIN SCHOOL OF
BUSINESS & INNOVATION**

Essay / Assignment Title: Home Loan Prediction

Programme title: Predictive Analytics and Machine Learning using Python

Name: Anusha Kelagina Peredelu

Year:2024

CONTENTS

CONTENTS.....	2
LIST OF FIGURES	3
LIST OF TABLES	4
INTRODUCTION.....	6
DATA PREPROCESSING	8
MODEL SELECTION AND TRAINING	21
MODEL EVALUATION	25
CONCLUSION	29
BIBLIOGRAPHY	30
APPENDIX.....	31

LIST OF FIGURES

Figure 1:Data Analysis Process	8
Figure 2:Dataset Information.....	9
Figure 3:Barplot of missing values in the dataset.....	10
Figure 4:Applying Median, KNN and Mode for columns to handle missing values	11
Figure 5:Outlier detection for numerical columns.....	12
Figure 6:Handling outlier using capping	13
Figure 7:Handling outliers by median imputation using IQR	13
Figure 8:Distribution of Numerical columns after handling outliers.....	14
Figure 9:Pie chart representing the distribution of the categorical features	15
Figure 10:Barplot for distribution of categorical features	16
Figure 11:Distribution of categorical features with respect to the Loan Status.....	17
Figure 12:Histogram for distribution of numeric features	18
Figure 13:Distribution of credit history	18
Figure 14:Income distribution by Gender and Education.....	19
Figure 15:Correlation matrix for numerical features	19
Figure 16:Data Splitting.....	22
Figure 17:Model training using Random Forest.....	23
Figure 18:Model training using Logistic Regression.....	23
Figure 19:Model training using KNN.....	24
Figure 20: Comparision of Random Forest, Logistic Regression and KNN	27
Figure 21:Model Deployment.....	28
Figure 22:Feature importance according to Random Forest.....	29

LIST OF TABLES

Table 1:Data Snapshot	9
Table 2:Performance metrics of Random Forest	25
Table 3: Performance metrics of Logistic Regression	26
Table 4: Performance metrics of K-Nearest Neighbors.....	26

Statement of compliance with academic ethics and the avoidance of plagiarism

I honestly declare that this dissertation is entirely my own work and none of its part has been copied from printed or electronic sources, translated from foreign sources and reproduced from essays of other researchers or students. Wherever I have been based on ideas or other people texts I clearly declare it through the good use of references following academic ethics.

(In the case that is proved that part of the essay does not constitute an original work, but a copy of an already published essay or from another source, the student will be expelled permanently from the postgraduate program).



Name and Surname (Capital letters):

ANUSHA KELAGINA PEREDELU

Date:2024/10/05

INTRODUCTION

Predictive analytics is the process of predicting what might happen in future based on what has happened in the past. This skill is called as cognitive skills through which the humans differ from other things. Analyzing each factor and learning from the past mistakes makes a person program the models. Once the model has been programmed the models can take decisions on what can be done in future. This analysis and process is known as predictive analysis in machine learning.

Machine learning (ML) is a term coined by Arthur Samuel in 1959. As the name suggests, it aims to infuse the learning capabilities of human in the form of an algorithm, model, architecture and/or techniques clubbed with data analytics, into a machine- computer. (Philomine Roseline. T,2024). This can also be said as a tool or a technique which is programmed and when given data the model learns and finally predictions are made.

Machine learning is classified into supervised, unsupervised and reinforcement/semi-supervised learning. Supervised learning deals with the data that is labelled data. Here the model is fed with input values and also the output values. The training data is trained on these input and output data until the predictions are made. Unsupervised learning deals with the unlabeled data. Here the model analyses the patterns and associations of the feature that is available and groups them accordingly. Semi-supervised learning deals with both labeled and unlabeled data. Here the model considers all the available patters, features and predicts the output. If there are noise in the data then it is tough for the model to predict the output accurately.

The main steps in predictive analytics are **Data preprocessing**, where the data is examined with the null values, checking for datatypes, checking for outliers etc. **Feature engineering**, where the trends, patters, extraction and selection of the features are made. **Model selection**, where the model is selected based on the data present and comparing with the type of model that best fits. **Training and Testing**, where the model is trained several times with training dataset and testing is made using testing data to predict the outcomes. **Evaluation**, where the model is evaluated and the accuracy of the model is found. **Deployment**, which is the final step performed once the better performing model has been choose.

These days predictive analytics has been used in many sectors like healthcare to predict the disease or the patient health based on his/her past health report. Business to predict the mistakes of past and learn from that and to build a higher strategy to reach certain goal. Social media were based on the user's patterns and trends the media is changing its configurations making people more engaging in social media. In Agriculture, Soil analytics, soil classification, intelligent harvesting, crop yield prediction, water management, livestock management, weed and disease detection, optimization of irrigation schedules etc. (Fatimetou Zahra Mohamed Mahmoud ,2017).

Overview of dataset:

In this assignment we are analyzing a dataset named Home Loan Approval. The problem statement is that Dream Housing Finance company deals in all home loans. They have a presence across all urban, semi-urban and rural areas. The customer first applies for a home loan after that company validates the customer's eligibility for a loan. The company wants to automate the loan eligibility process based on customer detail provided while filling out the online application form. These details are Gender, Marital Status, Education, Number of Dependents, Income, Loan Amount, Credit History and others. To automate this process, they have given a problem identifying the customer segments eligible for loan amounts to target these customers specifically.

The dataset is a labelled data consisting of input features which includes the customer details and the output feature which says whether the customer is eligible for loan or not. The features are Gender (Male, Female), Marital Status (Yes, No), Number of Dependents (0,1,2,3 or more) Self_Employed(Yes,No),(thousands),CoapplicantIncome(thousands),LoanAmount(tenthousands),Loan_Amount_Term(months),Credit_history(0,..1),Property_Area(Urban,semiurban,rural) and output feature Loan_Status (Y, N). We will follow the predictive analysis steps which are data preprocessing, feature engineering, model selection, Evaluating. We will process the data, handle the missing values, handle outliers. Perform exploratory data analysis where we are comparing multiple features and patterns and predicting the relationship between the inputs and the output variable. Select the model, split the data into train and test, fit the model and evaluate the model with the accuracy score.

DATA PREPROCESSING

Data analysis is the process of analysing the data and drawing conclusions out of them. This process includes 5 steps as shown in Figure 1, Collecting the data, cleaning, analysing, visualizing and finally evaluating the data.

Data Collection: The first step in data analysis is the process of collecting the data, Data can be collected from different sources online or different organizations from different sectors.

Data Cleaning: Data cleaning is the process of handling the dataset if there are null values present, redundant data, removing outliers etc.

Analyse Data: Analysing data includes analysing the trends, patterns from the features available in the data set. Train and test the data with machine learning models.

Visualizing Data: Visualizing data is the process of visualizing the relationship between the features and the target value to understand the dataset visually.

Evaluate Data: In This step we are evaluating the model that has been performed and drawing the conclusions out of the result.

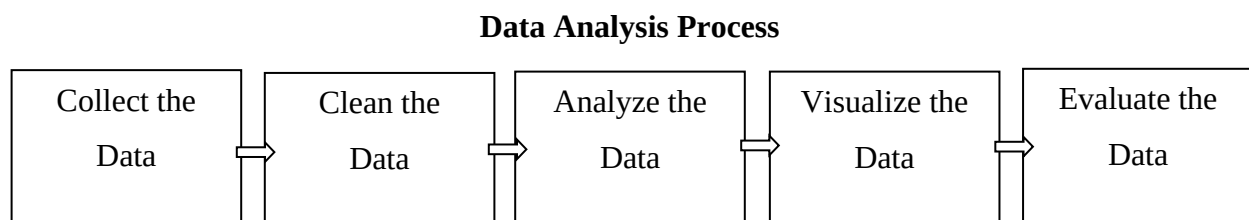


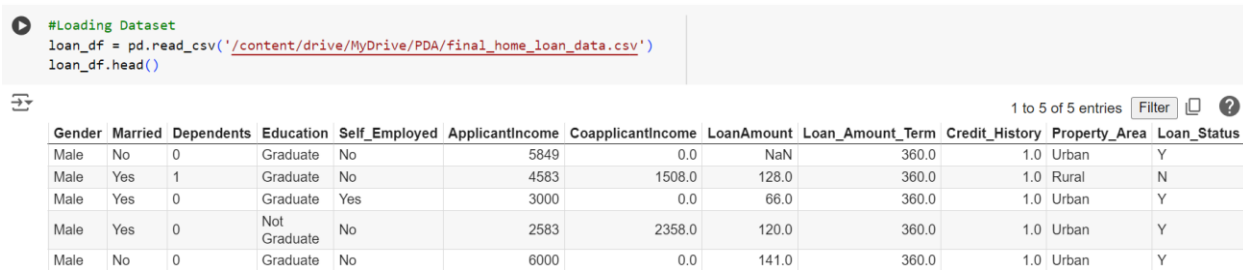
Figure 1:Data Analysis Process

Data preprocessing is an important step in the process of business behaviour analysis. This step mainly involves using techniques such as data cleaning, normalization, classification, and clustering to remove noise and redundant data, reduce irrelevant data, and improve data quality and availability (Yue shi,2023).

Loading the dataset:

The dataset is chosen from Kaggle and below is the first view of the dataset

```
#Loading Dataset
loan_df = pd.read_csv('/content/drive/MyDrive/PDA/final_home_loan_data.csv')
loan_df.head()
```



Gender	Married	Dependents	Education	Self_Employed	ApplicantIncome	CoapplicantIncome	LoanAmount	Loan_Amount_Term	Credit_History	Property_Area	Loan_Status
Male	No	0	Graduate	No	5849	0.0	NaN	360.0	1.0	Urban	Y
Male	Yes	1	Graduate	No	4583	1508.0	128.0	360.0	1.0	Rural	N
Male	Yes	0	Graduate	Yes	3000	0.0	66.0	360.0	1.0	Urban	Y
Male	Yes	0	Not Graduate	No	2583	2358.0	120.0	360.0	1.0	Urban	Y
Male	No	0	Graduate	No	6000	0.0	141.0	360.0	1.0	Urban	Y

Table 1:Data Snapshot

In the above Table 1, we have read the data using `pd.read_csv` and checking the head of the data set .This data set includes 12 columns in which 11 are input/feature columns where's 1 is the output column. Based on the inputs given we are calculating if the person is eligible for loan or not. The customer details include Gender of the person, married if the person is married or not, Dependents, how many dependents does the person have. Education, whether the person is graduate or not. Self_Employed if a person is self employed or not. It also includes applicantIncome, Income of the coapplicant, Loan amount that the person is requesting for, Loan amount term that the person is opting, Credit history of the person, Property_area is the area at which the person is willing to build a house. The outcome column Loan_status which has a value Y and N, which says if a person is eligible for loan or not.

```
[38] # Dataset Info
loan_df.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 981 entries, 0 to 980
Data columns (total 12 columns):
#   Column                Non-Null Count  Dtype  
---  -
0   Gender                 957 non-null   object  
1   Married                 978 non-null   object  
2   Dependents              956 non-null   object  
3   Education               981 non-null   object  
4   Self_Employed           926 non-null   object  
5   ApplicantIncome         981 non-null   int64   
6   CoapplicantIncome       981 non-null   float64  
7   LoanAmount              954 non-null   float64  
8   Loan_Amount_Term        961 non-null   float64  
9   Credit_History           902 non-null   float64  
10  Property_Area           981 non-null   object  
11  Loan_Status             981 non-null   object  
dtypes: float64(4), int64(1), object(7)
memory usage: 92.1+ KB
```

Figure 2:Dataset Information

In figure 2, we can see that the dataset includes both categorical and numerical values. 7 columns are object (Gender, Married etc) where 5 columns are numerical out of which one is integer

(ApplicantIncome) and the rest are floating point numbers. The above picture gives the structure of data frame which also includes missing values.

Data Cleaning

Data cleaning is one of the major steps in data analysis. This is important as the data contains many noises, null values, duplicates and other irrelevant values which will affect the analysis of the data. Hence cleaning of data is very important.

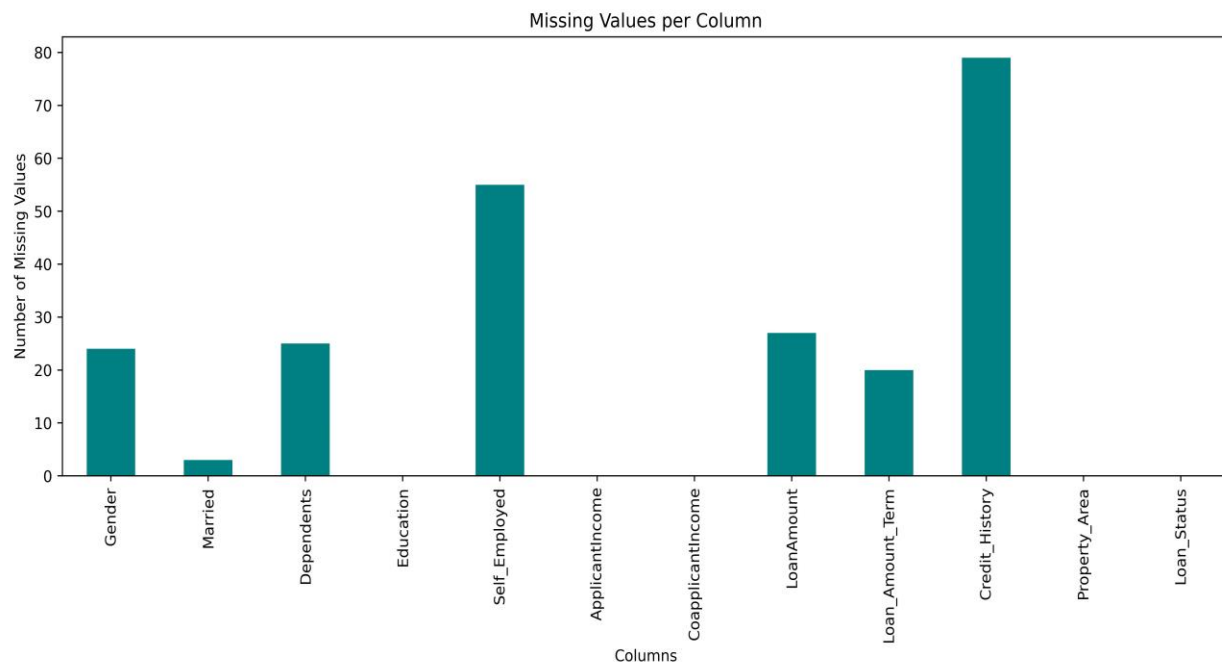


Figure 3: Barplot of missing values in the dataset

Figure 3, gives us the Barplot of the missing values in the dataset. As we see in the figure there are many null values in the columns Gender, Dependents, Self_Employed, LoanAmount, Loan_Amount_Term and Credit_History and a very few null values in Married column.

The missing or null values in columns like Gender, Married, Self_Employed, Credit_History are filled with mode of those specific columns. Mode is the value that appears frequently in the data. Applying mode to these columns will make the performance better. The missing values in column Dependents is filled with 0.

LoanAmount column is filled with median of that column. Median is the middle value in the entire column when it is sorted in ascending order. since LoanAmount is continuous variable applying median is the best choice as the missing values will not be dominated by extreme values present in that column.

The missing values and null values in Loan_Amount_Term is handled using **KNN Imputation**

The k -Nearest-Neighbours (kNN) is a non-parametric classification method. For a data record t to be classified, its k nearest neighbours are retrieved, and this forms a *neighbourhood of t* . Majority voting among the data records in the neighbourhood is usually used to decide the classification for t with or without consideration of distance-based weighting (Guo, Gongde & Wang, Hui & Bell, David & Bi, Yaxin, 2004).

KNN firstly understands the data points or neighbors, which is the other feature data point that is closest. Number of neighbors are denoted by K . For each missing value KNN searches for the values of other data point that are similar. Similarity is measured using distance metric (eg. Euclidean Distance). Once the KNN identifies the closest neighbor then it handles the missing value. If the feature is categorical then it finds mode, if the feature is numerical then it finds the average or mean of the values and fill the missing value (Figure 4).

```
#Imputing LoanAmount with median, Loan_Amount_Term with KNN, and Credit_History with mode
from sklearn.impute import SimpleImputer
from sklearn.neighbors import KNeighborsRegressor

# Impute LoanAmount with the median
loan_amount_imputer = SimpleImputer(strategy='median')
df['LoanAmount'] = loan_amount_imputer.fit_transform(df[['LoanAmount']])

# Impute Loan_Amount_Term using KNN
# Select features for KNN imputation
knn_features = ['ApplicantIncome', 'CoapplicantIncome', 'LoanAmount', 'Credit_History']
X_knn = df[knn_features].fillna(0) # Temporary fill of other NaNs

knn_imputer = KNeighborsRegressor(n_neighbors=5)
knn_imputer.fit(X_knn[df['Loan_Amount_Term'].notnull()], df.loc[df['Loan_Amount_Term'].notnull(), 'Loan_Amount_Term'])
df.loc[df['Loan_Amount_Term'].isnull(), 'Loan_Amount_Term'] = knn_imputer.predict(X_knn[df['Loan_Amount_Term'].isnull()])

# Impute Credit_History with mode
credit_history_imputer = SimpleImputer(strategy='most_frequent')
df['Credit_History'] = credit_history_imputer.fit_transform(df[['Credit_History']])

# Verify that no missing values remain
print(df[['LoanAmount', 'Loan_Amount_Term', 'Credit_History']].isnull().sum())
```

Figure 4: Applying Median, KNN and Mode for columns to handle missing values

Handling Outliers:

Handling outliers is very important in the process of data analysis. The main aim of handling Outlier is because of the inconsistency in the data which will lead to inappropriate results. Outlier analysis is a data mining step that detects data points, events, and/or findings that lie far away from other values of a dataset. (Iqbal H. Sarker, 2021)

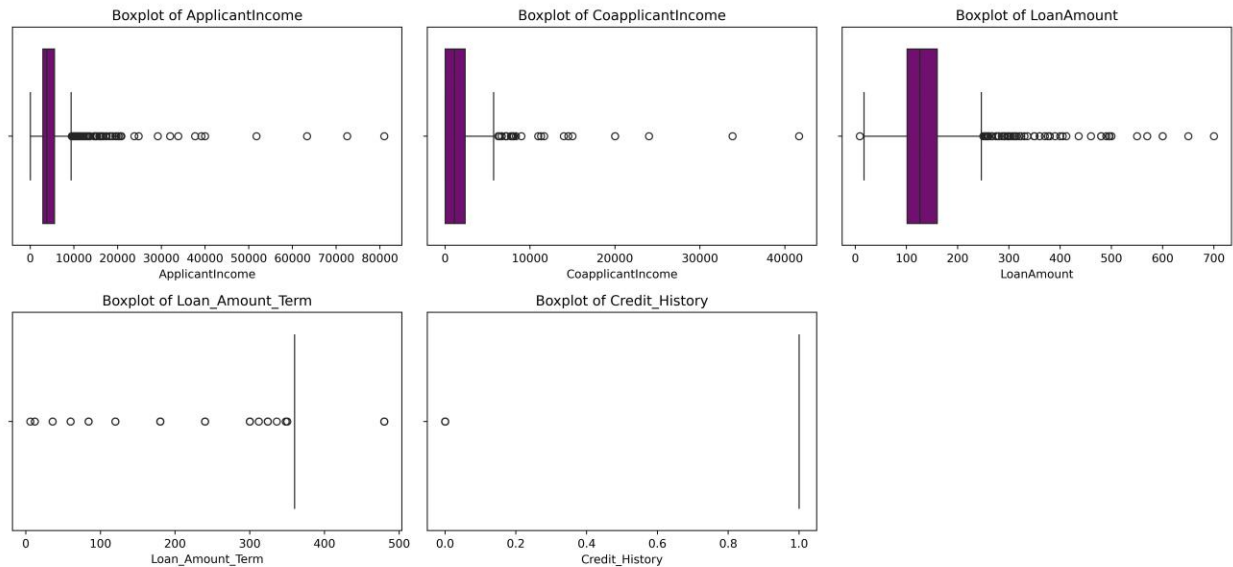


Figure 5: Outlier detection for numerical columns

The above figure (Figure 5) helps us visualize the outliers present in the numerical features. As we can see the columns ApplicantIncome, CoapplicantIncome and LoanAmount has a greater number of outliers and the column Loan_Amount_Term and Credit_History has a smaller number of outliers.

The methods that we have used here to handle outliers are **Capping** and **Median imputation using IQR**.

Capping is the method where the outliers are capped at a certain percentile limit. This method will not remove the values or the outliers completely instead it adjusts the value reducing influence on analysis.

cap_outlier is the function used to cap the extreme values. These functions have the parameters like column, lower_percentile and upper_percentile.

```

# Handling outliers in ApplicantIncome (using capping)
def cap_outliers(column, lower_percentile=0.01, upper_percentile=0.99):
    lower_limit = df1[column].quantile(lower_percentile)
    upper_limit = df1[column].quantile(upper_percentile)
    df1[column] = np.where(df1[column] > upper_limit, upper_limit, df1[column])
    df1[column] = np.where(df1[column] < lower_limit, lower_limit, df1[column])

# Cap the outliers in ApplicantIncome
cap_outliers('ApplicantIncome')

```

Figure 6: Handling outlier using capping

Figure 6, depicts handling outliers using capping method. Column is the column for which the outlier needs to be handled. lower_percentile is the lower percentile values used for capping (0.01 that's the 1st percentile value). upper_percentile is the upper percentile value used for capping (0.99 that is 99th percentile). Here the capping is made in such a way that the function caps or replaces any value that is below the lower percentile with 1st percentile value and the value above upper limit is replaced with the 99th percentile value. This process brings the value to a reasonable range.

```

[25] # 3. Handling outliers in LoanAmount (median imputation for outliers)
def replace_outliers_with_median(column):
    median_value = df1[column].median()
    iqr = df1[column].quantile(0.75) - df1[column].quantile(0.25)
    lower_bound = df1[column].quantile(0.25) - 1.5 * iqr
    upper_bound = df1[column].quantile(0.75) + 1.5 * iqr
    df1[column] = np.where((df1[column] < lower_bound) | (df1[column] > upper_bound), median_value, df1[column])

# Replace outliers in LoanAmount with the median
replace_outliers_with_median('LoanAmount')

```

Figure 7: Handling outliers by median imputation using IQR

Figure 7, Outliers in column LoanAmount is handled using median imputation using IQR. Median is the middle value of the entire column. The outliers are then found out using IQR method.

The IQR is a measure used to detect outliers. It is the range between the first quartile (Q1) and third quartile (Q3). Any data point below $Q1 - 1.5 \times IQR$ or above $Q3 + 1.5 \times IQR$ is considered an outlier.

$$IQR = Q3 - Q1$$

IQR method is preferred for general use due to its flexibility, especially in cases where the data does not conform to a normal distribution. (H P, Vinutha & Poornima, B. & Sagar, B. ,2018)

In this method the outliers are first checked using IQR method and then the values that are below lower_bound and that are above upper bound are then replaced using the median of that specific column.

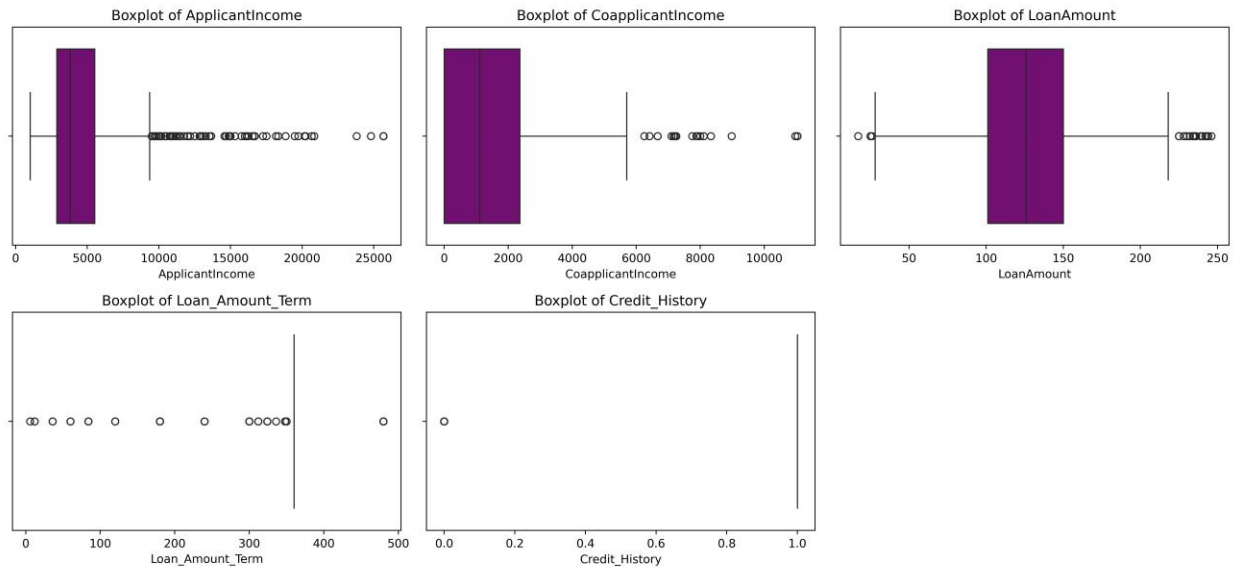


Figure 8: Distribution of Numerical columns after handling outliers

When comparing figure 5 and figure 8, we can see that the highest value in the ApplicantIncome column was 80000 which is now reduced to 25000, whereas the CoapplicantIncome was 40000 which is now clipped at 10000, LoanAmount was 700 which is now reduced to 250. Now the values in the dataset are normalized and are within the expected range.

Data Visualization

Data Visualization is the art and science of making data easy to understand and consume, for the end user. (Srivastava, Deepmala, 2023). The importance of data visualization in data analysis is more as it gives a visual picture of the distributions of the data and the relationships between input and output variables. There are many graphs in data visualization namely pie charts, bar graphs, count plots, line graphs, boxplot and many more. Here in this assignment, we will discuss the distribution of each feature, relationship between input and the output variables etc.

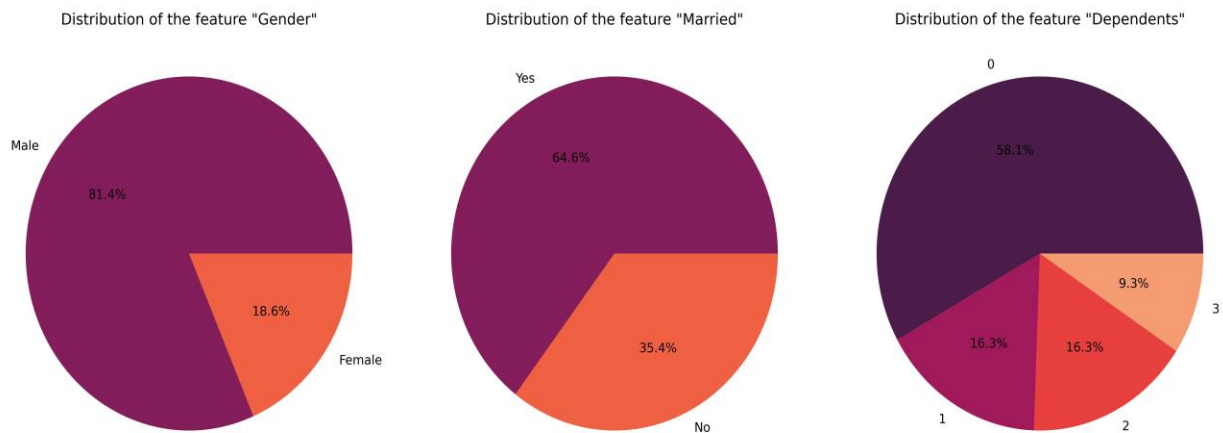


Figure 9: Pie chart representing the distribution of the categorical features

Figure 9 shows the distribution of the categorical features such as “Gender”, “Married” and “Dependents”. In the first chart, distribution of Gender, we can see that major portion (81.4%) of the applicants are Male whereas only 18.6% are Female applicants. From this we can say that Male applicants are more compared to female.

In the second graph we can see that 64.6% of the applicants are married whereas 35.4% of the people are not married. This suggests that married factor may affect the features like dependents or income. In the dependents graph we can see that about 58.1% of the people are with 0 dependents, 16.3% of the people have 1 or 2 dependents and 9.3% of the people have 3 or more dependents.

From these graphs we can say that the dataset is skewed in terms of gender and male applicants are more. Most of the applicants are married which might impact the Income and and coapplicant income. Meanwhile most of the applicants do not have dependents which will relate with income and loan approval.

The below Bar graph (Figure 10) depicts the distribution of categorical features such as “Education”, “Self_Employed” and “Property_Area”.

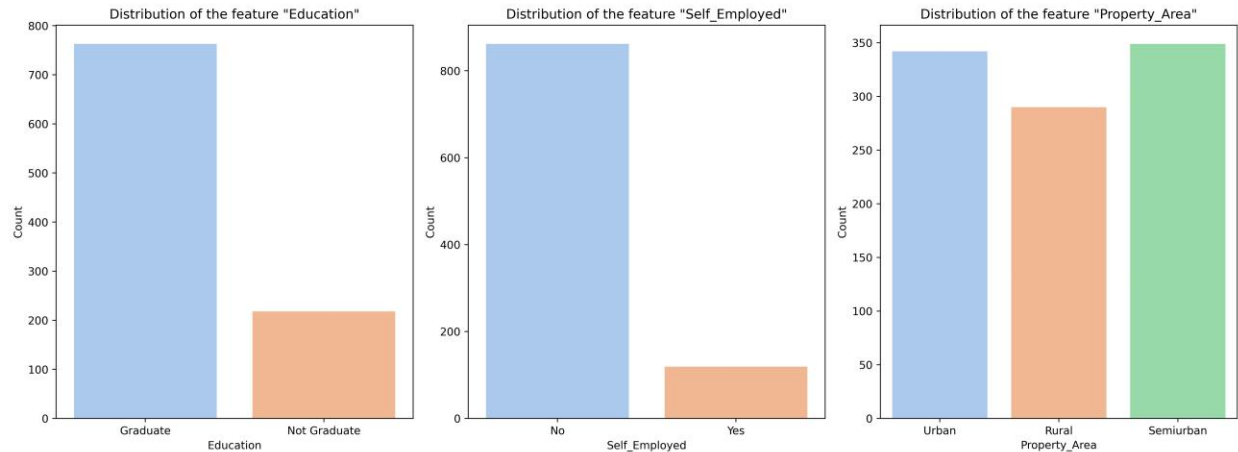


Figure 10: Barplot for distribution of categorical features

From the graph (Figure 10) we can see that in the Education sector Graduates are high indicating that most of the applicants have completed higher education. Smaller groups are non-Graduates with limited education, since most of them are graduates this factor can play an important factor in loan approval.

Higher number of applicants are not self-employed, but only few are self-employed. This suggests that the most of the people are salaried and may have chances of getting the loan. The property area for which they are applying the loan has balanced distribution among Rural, Urban and Semi-urban areas but Semiurban areas have slightly more applicants, followed closely by urban and rural areas, showing a mix of property types among applicants.

Figure 11, illustrate the distribution of various categorical features with respect to the Loan Status (whether the loan was approved "Y" or denied "N")

Gender vs Loan Status: The majority of the loan applicants are male in which 590 applications were approved and 209 are denied. Female applicants are less in which 127 are approved and 15 are denied. **Married vs Loan Status:** The overall count of married is high in which 483 applications were approved and 151 were denied. People who are unmarried have 234 applications approved and 113 are denied. **Dependents vs Loan Status:** Applicants with 0 dependents are more likely to get the loan approved rather than the applicants with dependents 1,2 or 3 and more. **Education vs Loan_Status:** Graduates are higher than non-graduates and we can see that the loan approval rate is higher in graduates when compared to non-graduates.

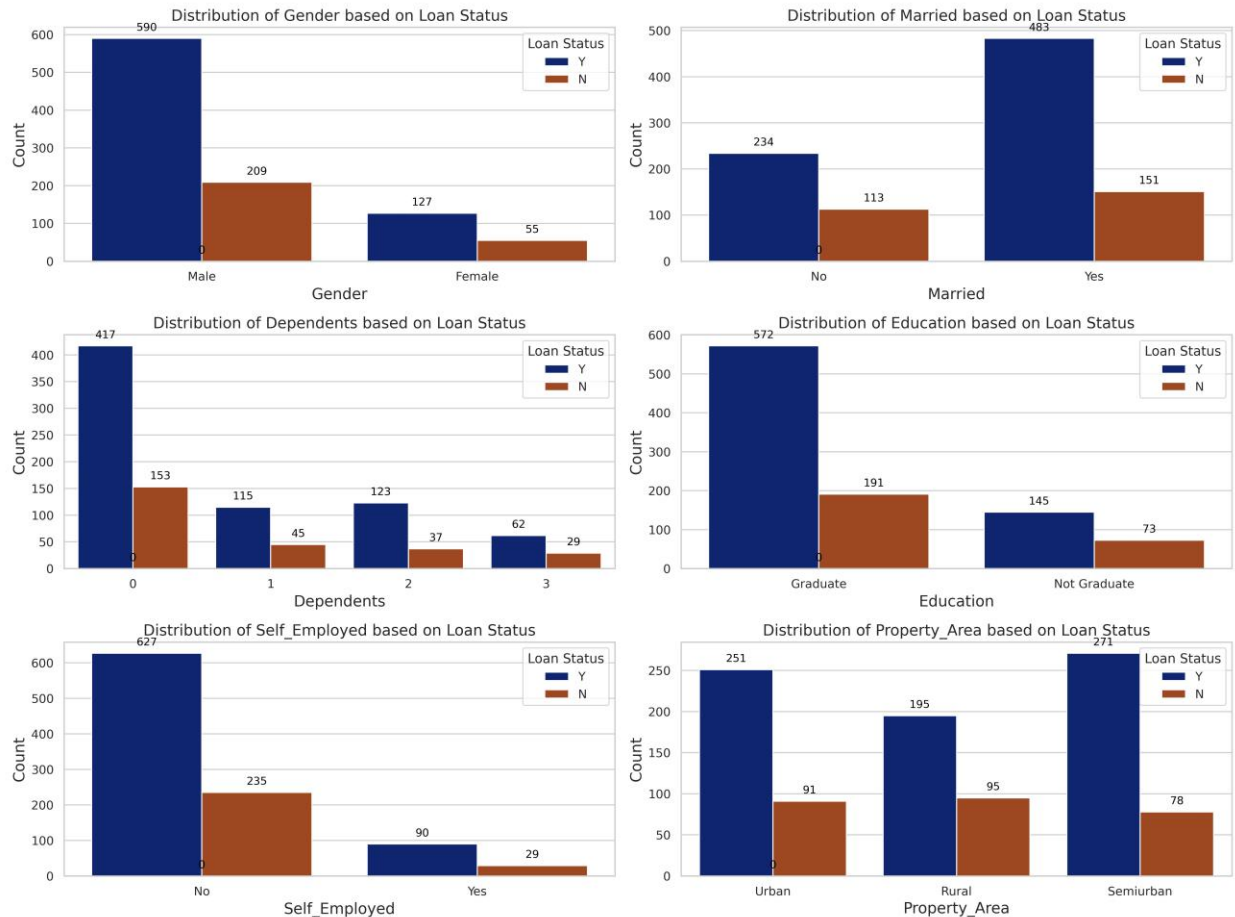


Figure 11: Distribution of categorical features with respect to the Loan Status

Self_Employed vs Loan_Status: A large number of applicants are not self-employed, out of which 627 were approved and 235 were denied. Self-employed applicants have a lower approval count, with 90 approved and 29 denied. **Property_Area vs Loan_Status:** Applicants from semi-urban areas have the highest approval rates, with 271 approved and 78 denied. Applicants from urban areas have the lowest loan approval rate, with 251 approvals and 91 rejections. Applicants from rural areas also face a lower approval rate, with 195 approvals and 95 rejections.

Gender, Education, and Property Area has greater impact on loan approval rates. Males, graduates, and applicants from semi-urban areas show higher chances of getting a loan approved. **Marital Status, Dependents** and **Self_Employed** also show influence, with married applicants who are self-employed and those without dependents being more likely to get approved.

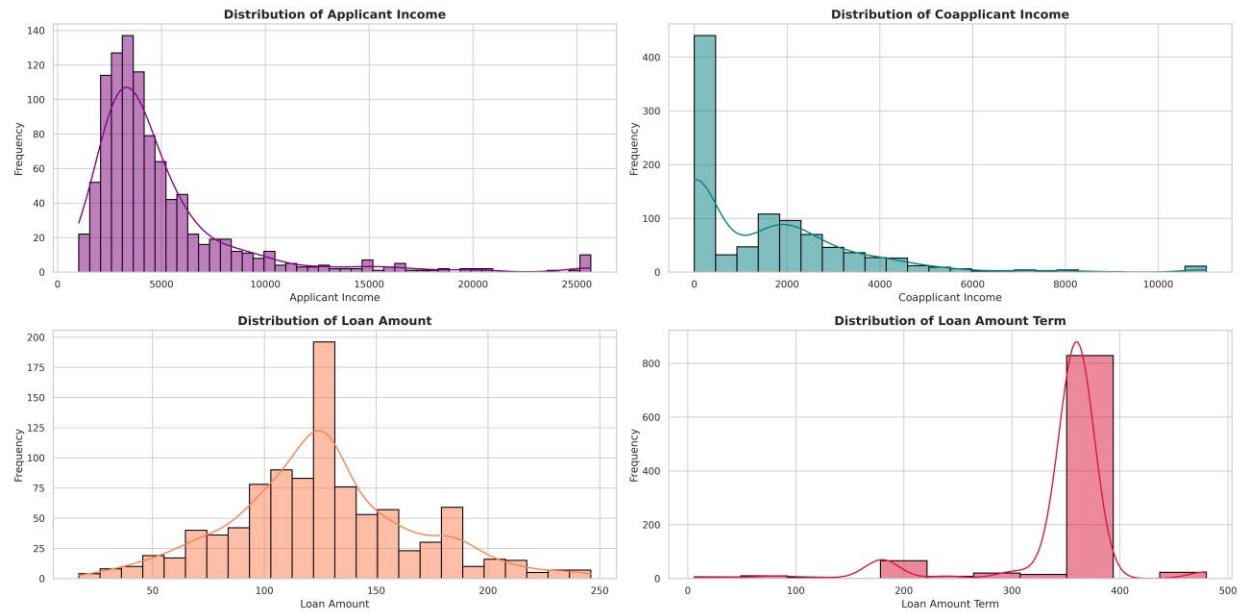


Figure 12:Histogram for distribution of numeric features

Figure 12, gives us the representation of distribution of numerical features of the dataset. When we look at the distribution of Applicant Income, we can see that the applicant income ranges from 1000 to 10000.Coapplicant income ranges from 0 to 4000.Loan amount ranges from 9 to 200 in multiples of 10,000 that is 90,000 to 20,00,000. Meanwhile, the Loan Amount Term ranges from 300 to 400 months.

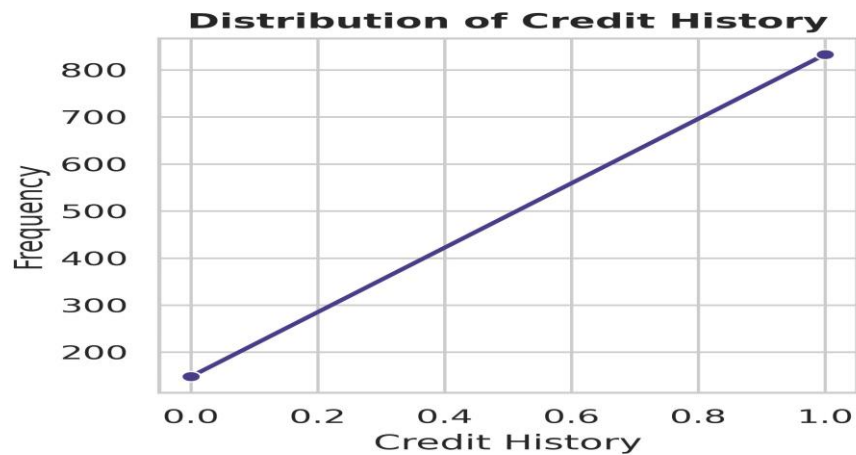


Figure 13:Distribution of credit history

Figure 13, tells us that the distribution of credit history of the applicants ranges from 0 to 1.

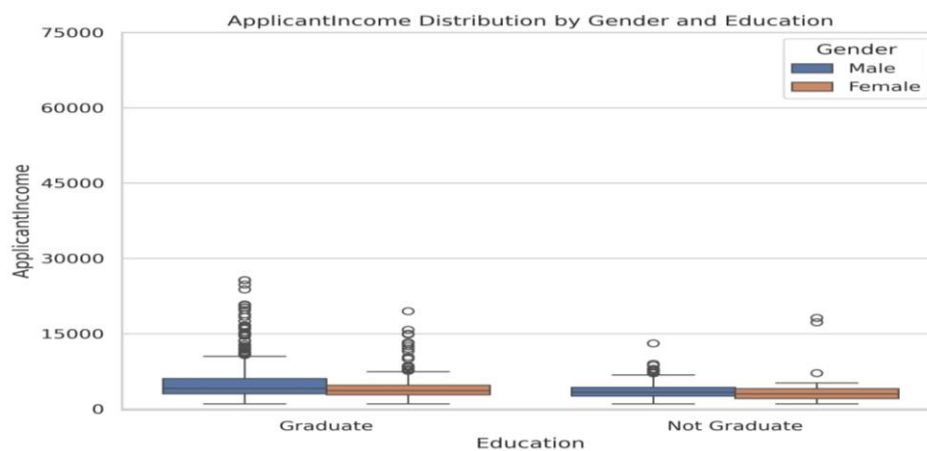


Figure 14: Income distribution by Gender and Education

Figure 14 visualizes the distribution of ApplicationIncome comparing Gender and Education features. In both Graduate and Non-Graduate categories, male applicants have higher incomes compared to female applicants. Non-Graduate applicants, regardless of gender, have relatively lower incomes. Comparing both educational categories, female applicants generally have a lower income. The variability in male applicant income is greater than that for female applicants.

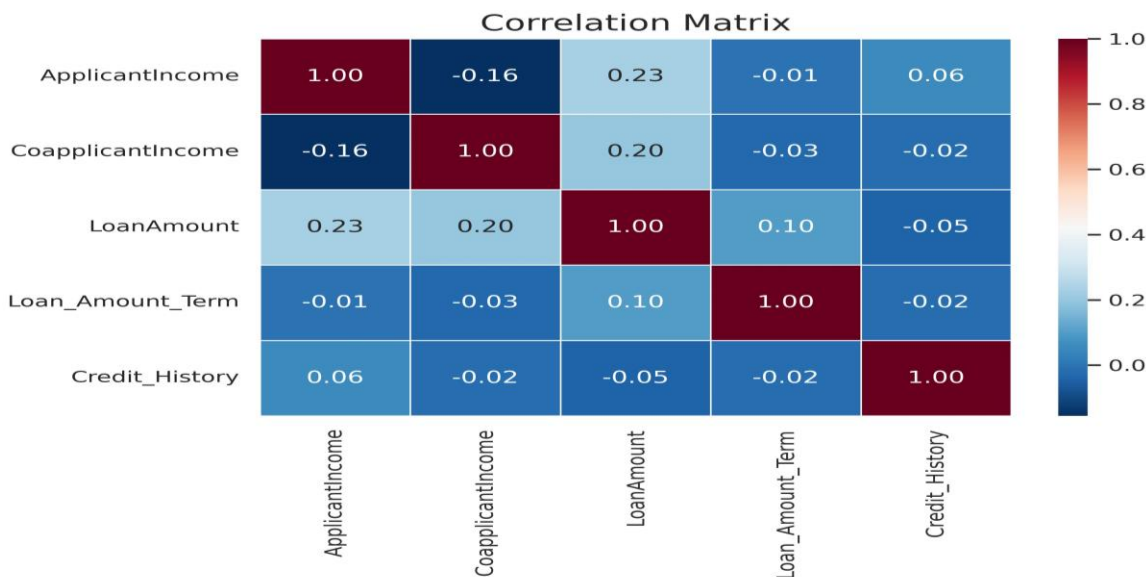


Figure 15: Correlation matrix for numerical features

In the above figure 15, we can see the correlation between the 5 variables ApplicationIncome, CoapplicantIncome, LoanAmount, Loan_Amount_Term and credit history. Correlation values

ranges from -1 to 1 where -1 is the negative correlation, 1 indicates the positive relationship and 0 indicates no linear relationship. ApplicantIncome and LoanAmount have a positive correlation (0.23), indicating that higher applicant incomes are associated with higher loan amounts. CoapplicantIncome also shows a positive correlation with LoanAmount (0.20), but the effect is slightly weaker than for ApplicantIncome. ApplicantIncome and CoapplicantIncome have a weak negative correlation (-0.16), suggesting that as one increases, the other slightly decreases. Credit_History has very weak correlations with all other variables, indicating that it is relatively independent of ApplicantIncome, Loan Amount, and Loan_Amount_Term variables. Loan_Amount_Term has zero correlations with all variables, suggesting that it does not have a linear relationship with the other financial variables.

Encoding

Encoding is the process of converting categorical columns into numerical. Since machine learning algorithms intake only numerical values it is necessary to convert the categorical values into numerical step before fitting the model.

In our dataset encoding is required for the categorical features available namely 'Gender', 'Married', 'Dependents', 'Education', 'Self_Employed', 'Property_Area'. We will be using label encoder to convert these categorical columns to numerical columns in further steps.

MODEL SELECTION AND TRAINING

The model that we are using today here in this assignment are **Random Forest (RF)**, **Logistic Regression (LR)** and **K-Nearest Neighbor (KNN)**

The **Random Forest** is an ensemble of unpruned classification or regression trees. Random Forest generates many classification trees and each tree is constructed by a different bootstrap sample from the original data using a tree classification algorithm. After the forest is formed, a new object that needs to be classified is put down each of the tree in the forest for classification. Each tree gives a vote that indicates the tree's decision about the class of the object. The forest chooses the class with the most votes for the object (Hasan M, Nasser, M, Ahmad S and Molla K, 2016)

Classification algorithms, such as **Logistic Regression**, are used in scenarios where data needs to be categorized into predefined classes (Shah, et al., 2020). These models work with dependent variables with binary values. These models use probability score as the predicted values of variables. It calculates the sum of input features and applies sigmoid function to map the sum obtained to probability of 0 and 1. If the probability is above 0.5 it predicts 1, else it predicts 0.

K-Nearest Neighbor, this technique finds the training set's most similar data points to generate predictions for classification. (Guo, Gongde & Wang, Hui & Bell, David & Bi, Yaxin, 2004). This technique will find the most similar data points from the training set to generate predictions of classification. It will check for the nearest data point from it. It finds the closest data point by measuring the distance using Euclidean distance. During training, it will not build the model, whereas it stores the data and makes decisions during prediction.

I have used these three models because Random Forest is good in handling non-linear data, it combines decision tree to improve accuracy and reduces over fitting. KNN is simple yet effective in classifying as it doesn't assume any underlying data distribution. Linear regression as it is the best model to use for the data with fewer features. Further will fit each model and analyse the result of Accuracy, Precision score, Recall, F1 score and so on.

Data Splitting

The splitting of data is one of the main steps to evaluate the performance of model. The data is split into Training set and Testing test. Training set is used to train the model where it understands the pattern in the data. Testing set is used to evaluate how the trained model generalizes the data and check the model's performance.

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder

# Encode categorical variables
df_encoded = df3.copy()

# List of columns to encode
categorical_cols = ['Gender', 'Married', 'Dependents', 'Education', 'Self_Employed', 'Property_Area']

# Apply Label Encoding
label_encoder = LabelEncoder()
for col in categorical_cols:
    df_encoded[col] = label_encoder.fit_transform(df_encoded[col])

# Step 2: Define Features and Target Variable
X = df_encoded.drop(columns=['Loan_Status'])
y = df_encoded['Loan_Status']

# Split the data into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)
```

Figure 16:Data Splitting

Figure 16, represents the data splitting process. Firstly, we have encoded the categorical columns using LabelEncoder. Since machine learning model accepts only numerical data encoding is necessary. To proceed we are defining feature and target variable. x defines the feature variable or the input variables and y define the Output variable that is Loan_Status. Then We use the train_test_split function from scikit-learn. Test Set Size, We chose a test size of 20%, meaning 80% of the data is used for training, and 20% is for testing. A random_state of 42 gives reproducibility of results by controlling the random shuffling of data. Stratified Split, To make sure that the proportion of each class in the target variable (y) is preserved in both the training and testing sets, we used the stratify parameter

Model Training

Model training is the process of feeding a machine learning algorithm with the data so that the model learns the patterns from the provided data.

Random Forest

```
▶ from sklearn.ensemble import RandomForestClassifier
   from sklearn.metrics import accuracy_score
   from sklearn.metrics import classification_report

   # Initialize the Random Forest model
   random_forest = RandomForestClassifier()

   # Train the model
   random_forest.fit(X_train, y_train)

   # Make predictions
   y_pred_random_forest = random_forest.predict(X_test)
```

Figure 17:Model training using Random Forest

In Figure 17, first we import the necessary libraries and then we proceed with initializing the random forest using `RandomForestClassifier()`. In order to train the model we fit the training dataset that is `X_train` and `y_train` into the model by calling `fit()` method. Finally we then predict the outcome on the test data by calling `predict()` function.

Logistic Regression

```
▶ from sklearn.linear_model import LogisticRegression

   Logistic_Regression=LogisticRegression()
   # Fit the model
   Logistic_Regression.fit(X_train, y_train)
   # Predict or evaluate
   y_pred_LR = Logistic_Regression.predict(X_test)
```

Figure 18:Model training using Logistic Regression

In Figure 18, first we import the necessary libraries and then we proceed with initializing the Logistic Regression using `LogisticRegression()`. In order to train the model we fit the training dataset that is `X_train` and `y_train` into the model by calling `fit()` method. Finally we then predict the outcome on the test data by calling `predict()` function with the `X_test`.

K-Nearest Neighbor

```
▶ from sklearn.neighbors import KNeighborsClassifier
   from sklearn.preprocessing import StandardScaler

   # Scale the features using StandardScaler
   scaler = StandardScaler()
   X_train_scaled = scaler.fit_transform(X_train)
   X_test_scaled = scaler.transform(X_test)

   # Initialize the K-Nearest Neighbors model (with k=5 by default)
   knn = KNeighborsClassifier(n_neighbors=5)

   # Train the model
   knn.fit(X_train_scaled, y_train)

   # Make predictions
   y_pred_knn = knn.predict(X_test_scaled)
```

Figure 19:Model training using KNN

In Figure 19, first we import the necessary libraries and then the features are scaled using StandardScaler this ensures the features have a mean value of 0 and standard deviation of 1 which makes all the features to equally contribute to the model. Then we proceed with initializing the Knearest Neighbor using KNeighborsClassifier() with n_neighbors=5. In order to train the model we fit the scaled dataset that is X_train_scaled and y_train into the model by calling fit () method. Finally we then predict the outcome on the test data by calling predict function predict() with the X_test_scaled.

MODEL EVALUATION

Model Evaluation

Model evaluation is the process of evaluating the performance of a machine learning model to understand how well it is performing.

Metrics chosen are Precision, recall, f1score and support. Precision gives us the information about the total predicted values how many were right. Recall is the measure of the actual correct values out of the correctly predicted values. F1 score is the harmonic mean of precision and recall and support is the number of occurrences of each instance in the data set.

Evaluation of Random Forest

Class	Precision	Recall	F1-Score	Support
0	0.90	0.70	0.79	53
1	0.90	0.97	0.93	144
Accuracy	-	-	89.85%	197
Macro Avg	0.90	0.84	0.86	197
Weighted Avg	0.90	0.90	0.89	197

Table 2:Performance metrics of Random Forest

This Table 2 gives us the evaluation for two classes that is Loan not approved(0) and Loan approved(1). In class 0 , precison value is 0.90 which indicates that the 90% of the data were preddicted correctly.Recall value is 0.70 which specifies that out of 90% prediction only 70 % of the data was actaully correct.F1 score is 0.79 which reflects the balance in metrics and there were 53 of the occurances of the class 0 in dataset.In class 1, precison value is 0.90 which indicates that the 90% of the data were preddicted correctly.Recall value is 0.97 which specifies that out of 90% prediction only 97 % of the data was actaully correct.F1 score is 0.93 which reflects the balance in metrics and there were 144 of the occurances of the class 0 in dataset.The overall accuracy of the model is **89.85% (90%)** indicating the model is performing very well.

Evaluation of Logistic Regression

Class	Precision	Recall	F1-Score	Support
0	0.94	0.64	0.76	53
1	0.88	0.99	0.93	144
Accuracy	-	-	89.34%	197
Macro Avg	0.91	0.81	0.85	197
Weighted Avg	0.90	0.89	0.89	197

Table 3: Performance metrics of Logistic Regression

This Table 3 gives us the evaluation for two classes that is Loan not approved(0) and Loan approved(1). In class 0 , precison value is 0.94 which indicates that the 94% of the data were precdicted correctly.Recall value is 0.64 which specifies that out of 94% prediction only 64 % of the data was actaully correct.F1 score is 0.76 which reflects the balance in metrics and there were 53 of the occurances of the class 0 in dataset.In class 1, precison value is 0.88 which indicates that the 88% of the data were precdicted correctly.Recall value is 0.99 which specifies that out of 88% prediction only 99 % of the data was actaully correct.F1 score is 0.93 which reflects the balance in metrics and there were 144 of the occurances of the class 0 in dataset.The overall accuracy of the model is **89.34% (89%)** indicating the model is performing quite good.

Evaluation of KNN

Class	Precision	Recall	F1-Score	Support
0	0.83	0.66	0.74	53
1	0.88	0.95	0.92	144
Accuracy	-	-	87.31%	197
Macro Avg	0.86	0.81	0.83	197
Weighted Avg	0.87	0.87	0.87	197

Table 4: Performance metrics of K-Nearest Neighbors

Table 4, gives us the evaluation for two classes that is Loan not approved(0) and Loan approved(1). In class 0 , precison value is 0.83 which indicates that the 83% of the data were precdicted correctly.Recall value is 0.66 which specifies that out of 83 prediction only 66 % of the data was actaully correct.F1 score is 0.74 which reflects the balance in metrics and there were 53 of the occurances of the class 0 in dataset.In class 1, precison value is 0.88 which indicates that the 88% of the data were precdicted correctly.Recall value is 0.95 which specifies that out of 88% prediction only 95 % of the data was actaully correct.F1 score is 0.92 which reflects the balance in metrics and there were 144 of the occurances of the class 0 in dataset.The overall accuracy of the model is **87.31% (87%)** indicating the model performance is comparitively low.

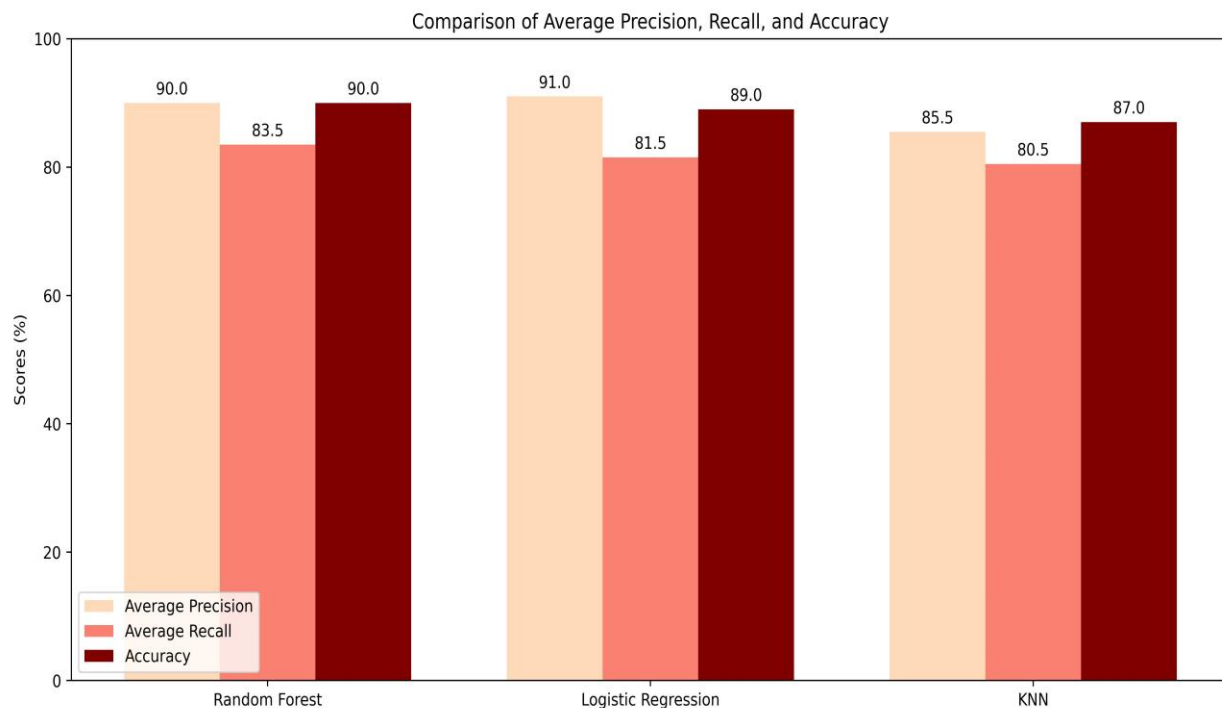


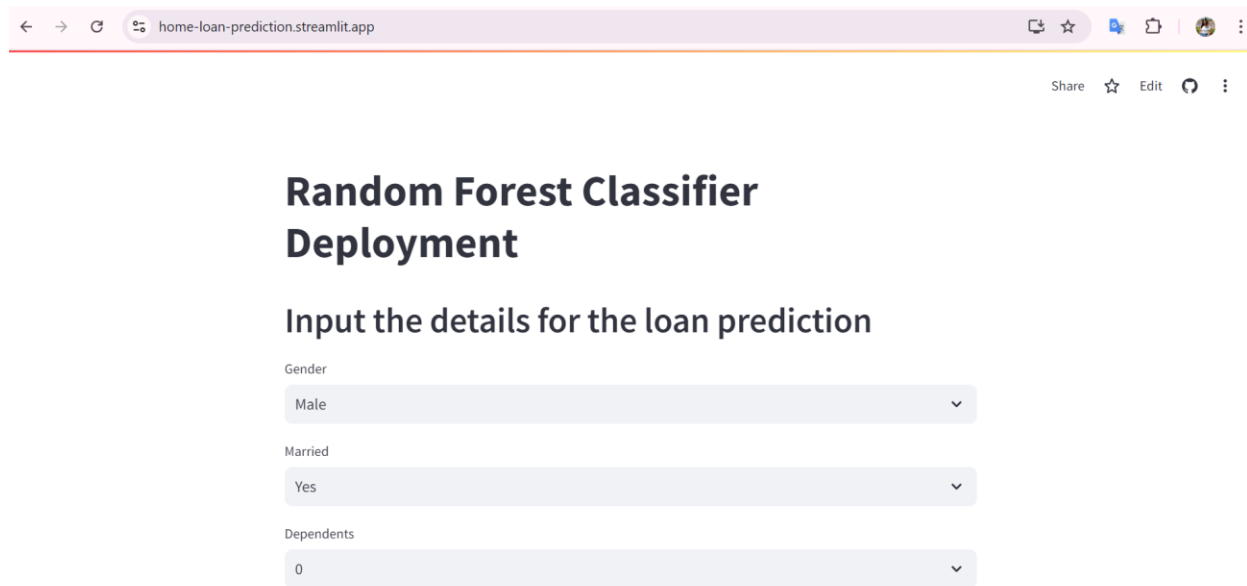
Figure 20: Comparision of Random Forest, Logistic Regression and KNN

Figure 20 gives us the comparision between three models that are “RandomForest”, “LogisticRegression” and “KNN”.As we look at the graph the average precision value that is the ability of the model random forest to predict the correct values is 90% whereas that of LogisticRegression is 91% which is quite high and that of KNN is comparitively low at 85.5%. Aaverage of Recall value that classifying the Loan status (Approved or Not approved) correctly is 83.5% in random forest , 81.5% in logistic regression and that of KNN is 80.5%. Accuracy,the

overall ratio of correctly predicted instances in Logistic Regression and KNN is 89% and 87% respectively which is low when compared to the accuracy of Random Forest that is 90%.

Hence, we can say that **Random Forest** stands out as the best model by achieving the highest overall accuracy and good precision and recall values for both the classes.

Model Deployment



The screenshot shows a web browser window with the address bar displaying 'home-loan-prediction.streamlit.app'. The page title is 'Random Forest Classifier Deployment'. Below the title, the instruction 'Input the details for the loan prediction' is shown. There are three dropdown menus for input: 'Gender' with 'Male' selected, 'Married' with 'Yes' selected, and 'Dependents' with '0' selected. The browser's address bar and navigation icons are visible at the top.

Figure 21:Model Deployment

The above figure 21 shows the model deployed into a web application where the user can give the details of loan prediction and predict whether he/she is eligible for Loan or not.

Please find the link to access the website: <https://home-loan-prediction.streamlit.app/>

CONCLUSION

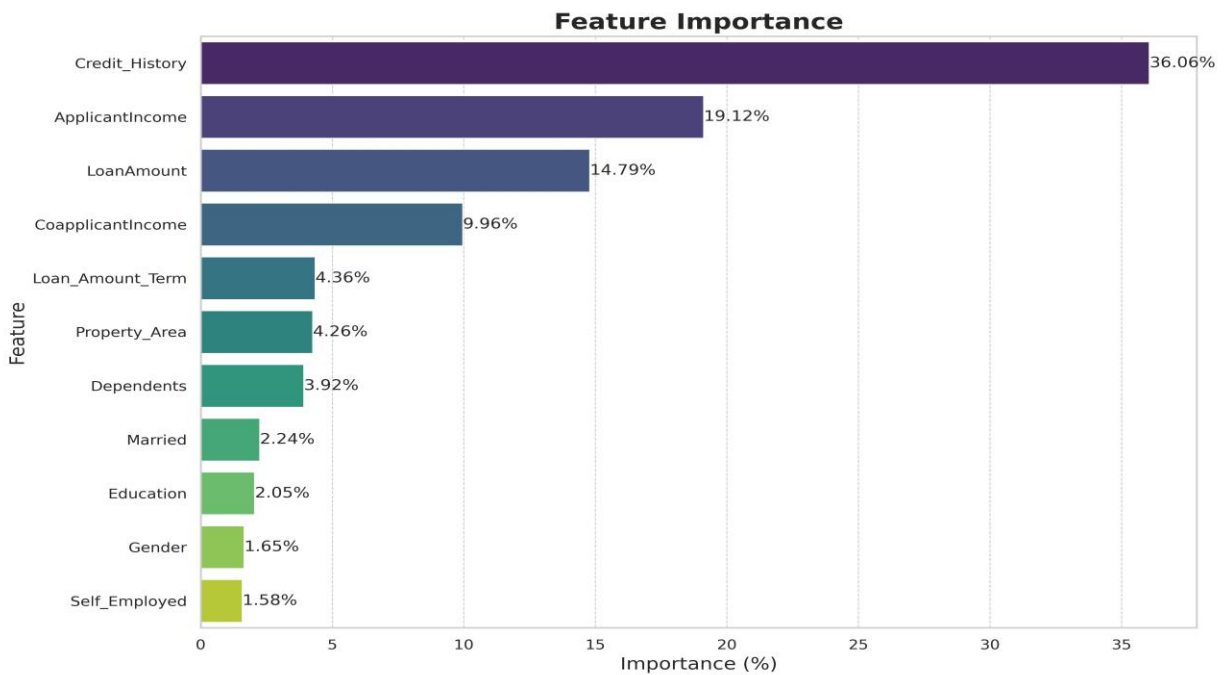


Figure 22: Feature importance according to Random Forest

The above figure 22 visualizes the feature importance in predicting loan approval based on a Random Forest model. The main feature that influences Loan Approval is **Credit_History** with 36.06% importance. This tells us that a person with highest credit history score is more likely to get the loan approved. The second feature that has impact is **ApplicantIncome** with 19.12% importance. If a person has higher income he/she is more likely to get the loan approved. **Loan amount** also influences the decision of approving the loan at 14.79%. **Coapplicant income** also has greater impact on the loan being approved. On the other hand **Loan_Amount_term**, **Property_Area** and **Dependents** feature have smaller influence each below 5% importance. **Married**, **Education**, **Gender**, and **Self_Employed** are at the least which are with 2% importance or below.

From the above observation we can conclude that financial factors like **Credit_History** and **ApplicantIncome** are the important features rather than personal details in predicting loan status.

BIBLIOGRAPHY

Philomine Roseline. T(2024) *Machine Learning and Predictive Analytics – A Paradigm Shift in Decision Making*, Research Scholar, Department of Computer Science, Bishop Heber College (Autonomous), Tiruchirappalli

Fatimetou Zahra Mohamed Mahmoud (May 2017), “*The Application Of Predictive Analytics: Benefits, Challenges And How It Can Be Improved*”, International Journal of Scientific and Research Publications, Volume 7, Issue 5.

Yue shi(2023)-*Application of Improved Linear Regression Algorithm*, University of Sheffield, Sheffield, England, UK

Guo, Gongde & Wang, Hui & Bell, David & Bi, Yaxin. (2004). *KNN Model-Based Approach in Classification*.

Iqbal H. Sarker (2021)- *Data Science and Analytics: An Overview from Data-Driven Smart Computing, Decision-Making and Applications Perspective*, Springer Nature Singapore Pte Ltd

H P, Vinutha & Poornima, B. & Sagar, B. (2018). *Detection of Outliers Using Interquartile Range Technique from Intrusion Dataset*. 10.1007/978-981-10-7563-6_53.

Srivastava, Deepmala. (2023). *An Introduction to Data Visualization Tools and Techniques in Various Domains*. International Journal of Computer Trends and Technology. 71. 125-130. 10.14445/22312803/IJCTT-V71I4P116.

Hasan, M, Nasser, M, Ahmad, S. and Molla, K. (2016) *Feature Selection for Intrusion Detection Using Random Forest*. Journal of Information Security, 7, 129-140.

Shah, K., Patel, H., Sanghvi, D., & Shah, M. (2020). *A comparative analysis of logistic regression, random forest and KNN models for the text classification*. Augmented Human Research, 5, 1-16.

APPENDIX

The code and additional analysis can be found in Google Collab Notebook:

https://colab.research.google.com/drive/1wSOAa6lf5SHcDX5u4ocl7_1rEquYk_KY?usp=sharing